CI/CD Pipeline for Scientific Calculator

Rohitangshu Bose
Enrollment No: MT2025106

GitHub Repository: github.com/Rohitangshu2026/scientific-calculator-ci-cd

1 Introduction

Continuous Integration and Continuous Deployment (CI/CD) pipelines automate the build, testing, packaging and deployment of software applications. In this project, a CI/CD pipeline is implemented for a Java-based Scientific Calculator application using the following tools:

- GitHub for version control
- Jenkins for CI/CD automation
- Maven for project build
- Docker for containerization
- DockerHub as container registry
- Ansible for automated deployment

Whenever a developer pushes code to the GitHub repository, Jenkins automatically triggers the pipeline to build, test, package and deploy the application.

2 CI/CD Pipeline Architecture

The following diagram shows the overall CI/CD workflow implemented in this project.

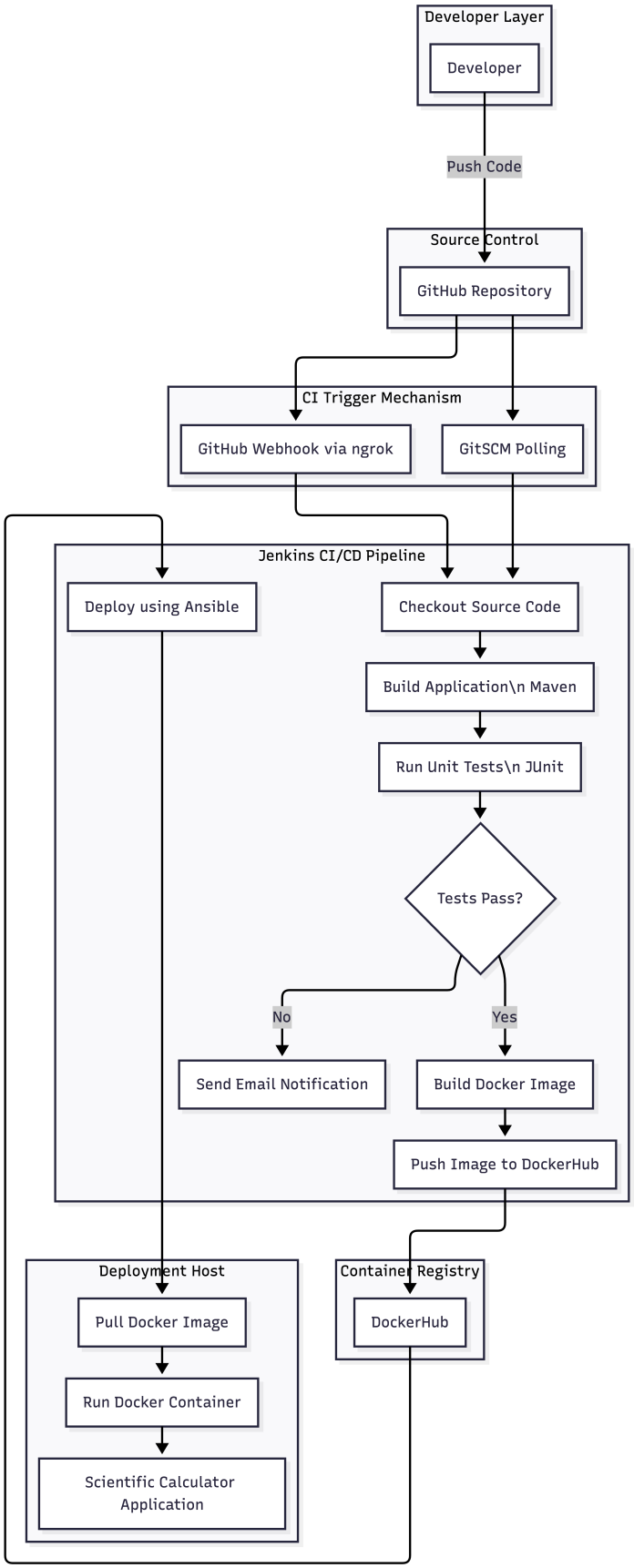
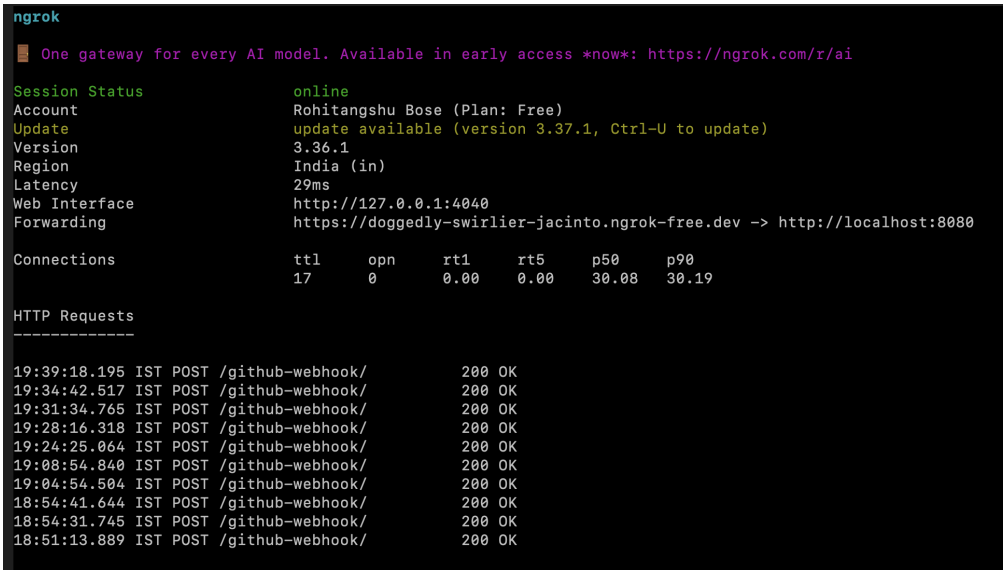


Figure 1: CI/CD Pipeline Architecture

The pipeline ensures that only successfully tested builds are packaged and deployed, preventing faulty code from reaching production environments.

3 GitHub Webhook Trigger

The Jenkins pipeline is automatically triggered when code is pushed to the GitHub repository. Since Jenkins runs locally, ngrok is used to expose the local Jenkins server to the internet so GitHub can deliver webhook events.



```
ngrok

 One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status                online
Account                       Rohitangshu Bose (Plan: Free)
Update                        update available (version 3.37.1, Ctrl-U to update)
Version                       3.36.1
Region                        India (in)
Latency                       29ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://doggedly-swirlier-jacinto.ngrok-free.dev -> http://localhost:8080

Connections                   ttl     opn     rt1     rt5     p50     p90
                              17      0       0.00    0.00    30.08   30.19

HTTP Requests
-------------

19:39:18.195 IST POST /github-webhook/          200 OK
19:34:42.517 IST POST /github-webhook/          200 OK
19:31:34.765 IST POST /github-webhook/          200 OK
19:28:16.318 IST POST /github-webhook/          200 OK
19:24:25.064 IST POST /github-webhook/          200 OK
19:08:54.840 IST POST /github-webhook/          200 OK
19:04:54.504 IST POST /github-webhook/          200 OK
18:54:41.644 IST POST /github-webhook/          200 OK
18:54:31.745 IST POST /github-webhook/          200 OK
18:51:13.889 IST POST /github-webhook/          200 OK
```

Figure 2: ngrok Tunnel exposing Jenkins to GitHub

GitHub sends POST requests to the Jenkins webhook endpoint whenever a push event occurs.

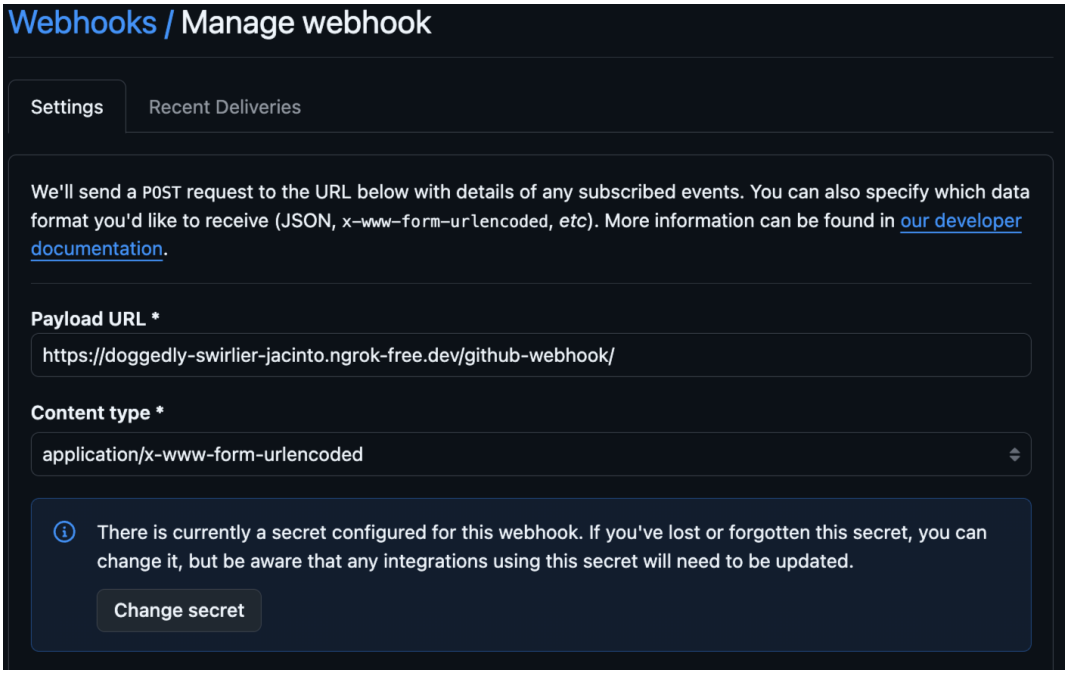


Figure 3: GitHub Webhook Configuration

4 Jenkins Pipeline Stages

The CI/CD pipeline is defined using a Jenkinsfile written in Declarative Pipeline syntax. Each stage performs a specific task in the build and deployment lifecycle.

4.1 Checkout Source Code

This stage retrieves the latest code from the GitHub repository.

```
stage('Checkout Source Code') {
    steps {
        git branch: 'main',
            url: "${GITHUB_REPO}"
    }
}
```

Figure 4: Checkout Source Code Stage

4.2 Build Application

The project is compiled using Maven.

```
stage('Build Application') {
    steps {
        sh 'mvn clean package -DskipTests'
    }
}
```

Figure 5: Build Application Stage

4.3 Run Unit Tests

JUnit tests verify the correctness of the calculator operations.

```
stage('Run Unit Tests') {
    steps {
        sh 'mvn test'
    }
}
```

Figure 6: Run Unit Tests Stage

If any test fails, the pipeline stops immediately.

4.4 Verify Build Artifact

This stage verifies that the compiled JAR file exists in the build output directory.

```
stage('Verify Build Artifact') {
    steps {
        sh 'ls -lh target/'
    }
}
```

Figure 7: Verify Build Artifact Stage

4.5 Build Docker Image

Once all tests pass, Jenkins builds a Docker image containing the application.

```
stage('Build Docker Image') {
    steps {
        sh "docker build -t ${DOCKER_IMAGE} ."
    }
}
```

Figure 8: Build Docker Image Stage

4.6 Push Docker Image to DockerHub

The Docker image is pushed to DockerHub so it can be used during deployment. The image can be accessed at:

hub.docker.com/repository/docker/rohitangshu/scientific-calculator

```
stage('Push Docker Image to DockerHub') {

    steps {

        withCredentials([usernamePassword(
                credentialsId: 'DockerHubCred',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
        )]) {

            sh '''
            echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin

            docker tag scientific-calculator $DOCKER_USER/scientific-calculator:latest

            docker push $DOCKER_USER/scientific-calculator:latest
            '''
        }
    }
}
```

Figure 9: Push Docker Image to DockerHub

DockerHub credentials are securely stored inside Jenkins.

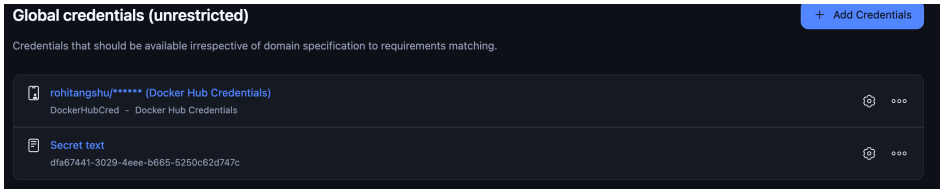


Figure 10: DockerHub Credentials Stored in Jenkins

4.7 Deployment Using Ansible

Deployment is automated using an Ansible playbook.

```
    stage('Deploy Using Ansible') {
        steps {
            sh 'ansible-playbook -i inventory deploy.yml'
        }
    }

}
```

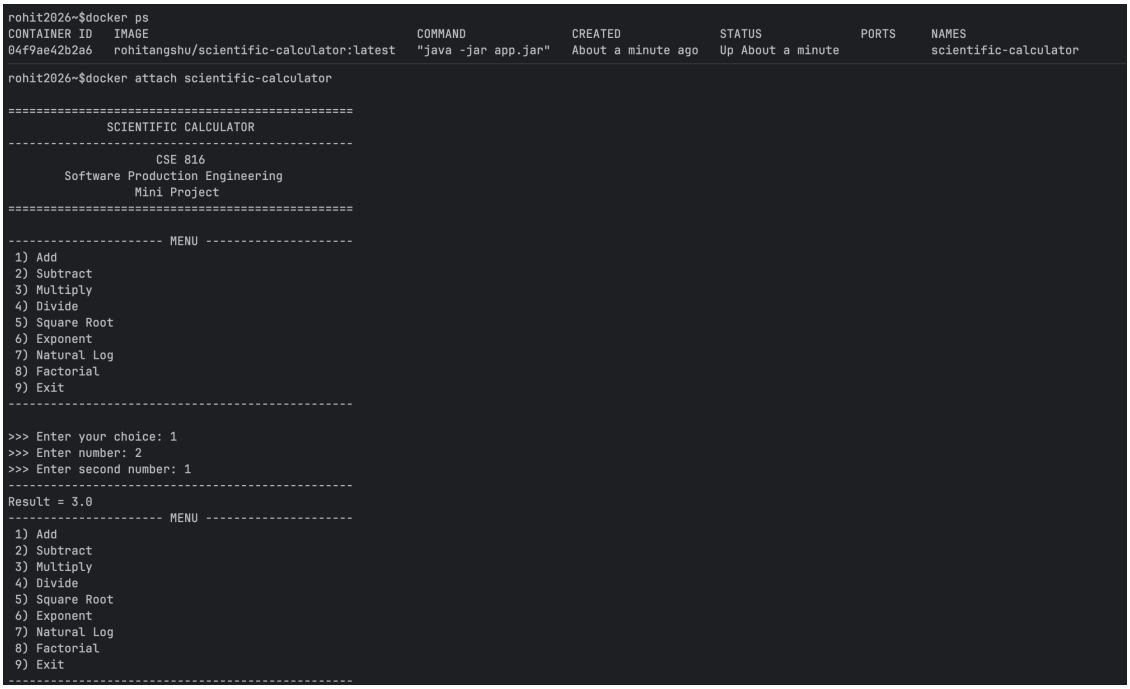
Figure 11: Deployment Using Ansible

The playbook performs the following actions:

- Pull latest Docker image
- Stop running container
- Remove old container
- Run new container

5 Application Running in Docker Container

After deployment, the scientific calculator application runs inside a Docker container.



```
rohit2026~$docker ps
CONTAINER ID   IMAGE                                       COMMAND               CREATED              STATUS              PORTS     NAMES
04f9ae42b2a6   rohitangshu/scientific-calculator:latest    "java -jar app.jar"   About a minute ago   Up About a minute             scientific-calculator

rohit2026~$docker attach scientific-calculator

==================================================
              SCIENTIFIC CALCULATOR
--------------------------------------------------
                     CSE 816
        Software Production Engineering
                   Mini Project
==================================================

---------------------- MENU ----------------------
 1) Add
 2) Subtract
 3) Multiply
 4) Divide
 5) Square Root
 6) Exponent
 7) Natural Log
 8) Factorial
 9) Exit
--------------------------------------------------

>>> Enter your choice: 1
>>> Enter number: 2
>>> Enter second number: 1
--------------------------------------------------
Result = 3.0
---------------------- MENU ----------------------
 1) Add
 2) Subtract
 3) Multiply
 4) Divide
 5) Square Root
 6) Exponent
 7) Natural Log
 8) Factorial
 9) Exit
--------------------------------------------------
```

Figure 12: Scientific Calculator Running Inside Docker Container

6 Pipeline Failure Demonstration

To demonstrate the failure handling capability of the CI/CD pipeline, a logical error was intentionally introduced in the square root function in **BUILD NUMBER 18**. Instead of computing the square root, the function returned the input number.

```
1 file changed  +1 -4  lines changed                                   Search within code

src/main/java/org/example/Calculator.java                                          +1 -4

@@ -23,10 +23,7 @@ public static double divide(double num1, double num2){
 23      }                                                          23      }
 24                                                                 24
 25      public static double squareRoot(double num){               25      public static double squareRoot(double num){
-26          if(num < 0){                                           26+         return num;   // intentionally wrong to test email notification when tests fail
-27              throw new ArithmeticException("Square root of negative number");
-28          }
-29          return Math.sqrt(num);
 30      }                                                          27      }
 31                                                                 28
 32      public static double exponent(double base, double exponent){  29      public static double exponent(double base, double exponent){
```

Figure 13: Commit Introducing Logical Error

When Jenkins executed the pipeline for this commit, the unit tests failed.

Figure 14: Pipeline Failure in Jenkins

After the failure, Jenkins automatically sent an email notification to the developer.

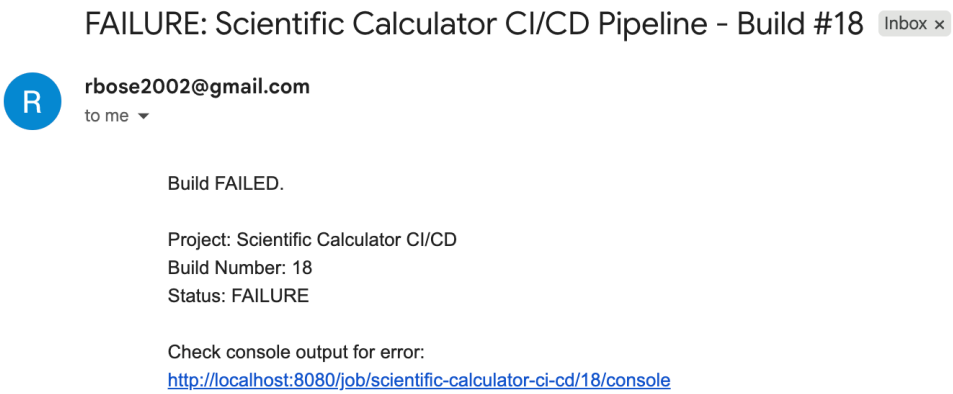


Figure 15: Email Notification for Failed Build

This demonstrates that the CI/CD pipeline successfully detects errors and alerts developers immediately.

7 Conclusion

This project demonstrates the implementation of a complete CI/CD pipeline for a Java-based Scientific Calculator application. The pipeline integrates GitHub, Jenkins, Docker and Ansible to automate the build, test, packaging and deployment process. Automated testing ensures that only verified code is deployed. Failure detection and email notifications enable

developers to quickly identify and fix issues. The pipeline improves reliability, automation and deployment efficiency.